

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software

TA — Tarea de práctica fuera de clases

Extender proyecto ASP.NET Core MVC

MiPrimerMVC

Asignatura: Aplicaciones Web

Estudiante: Zaida Melissa Taípe Mora

Docente: Ing. Guerrero Ulloa Gleiston Cicerón

Periodo Académico: 2026–2027 PPA

Quevedo, Ecuador

18 de junio de 2026

<https://github.com/Zaida-tm18/MiPrimerMVCExtendido.git>

1. Objetivo

Extender el proyecto MiPrimerMVC, de modo que el módulo de Productos pase de un simple registro de alta (Create) a un flujo más completo: listado con acciones por fila, eliminación (Delete) con pantalla de confirmación y protección contra ataques CSRF, validación de datos tanto en el cliente como en el servidor, control de versiones mediante una rama dedicada y un README actualizado con las instrucciones de ejecución.

2. Punto de partida: proyecto base

El proyecto base MiPrimerMVC ya contaba con: un modelo Producto con reglas de validación mediante Data Annotations (Required, StringLength, Range, EmailAddress), un ProductoController con las acciones Index (listado) y Create (alta de productos, GET/POST), y vistas Razor que usan tag helpers (asp-for, asp-validation-for, asp-validation-summary). Sobre esa base se desarrollaron las siguientes cinco extensiones, una por cada directriz de la tarea.

3. Tabla HTML con lista de productos

La vista Index.cshtml ya presentaba los productos registrados en una tabla HTML (Bootstrap table-striped) cuando la lista no está vacía, o un mensaje informativo cuando aún no hay registros. Para esta tarea se añadió una columna adicional de Acciones, con un botón Eliminar por cada fila que enlaza a la nueva pantalla de confirmación de borrado, usando el tag helper asp-route-id para pasar el identificador del producto.

```
<table class="table table-striped">
  <thead>
    <tr>
      <th>Id</th><th>Nombre</th><th>Descripcion</th>
      <th>Precio</th><th>Stock</th><th>Correo</th>
      <th>Acciones</th>
    </tr>
  </thead>
  <tbody>
    @foreach (var item in Model)
    {
      <tr>
        <td>@item.Id</td>
        <td>@item.Nombre</td>
        <td>@item.Descripcion</td>
        <td>@item.Precio.ToString("C")</td>
        <td>@item.Stock</td>
        <td>@item.CorreoProveedor</td>
        <td>
          <a asp-action="Delete" asp-route-id="@item.Id"
            class="btn btn-sm btn-danger">Eliminar</a>
        </td>
      </tr>
    }
  </tbody>
</table>
```

Listado 1: Views/Producto/Index.cshtml — columna Acciones agregada

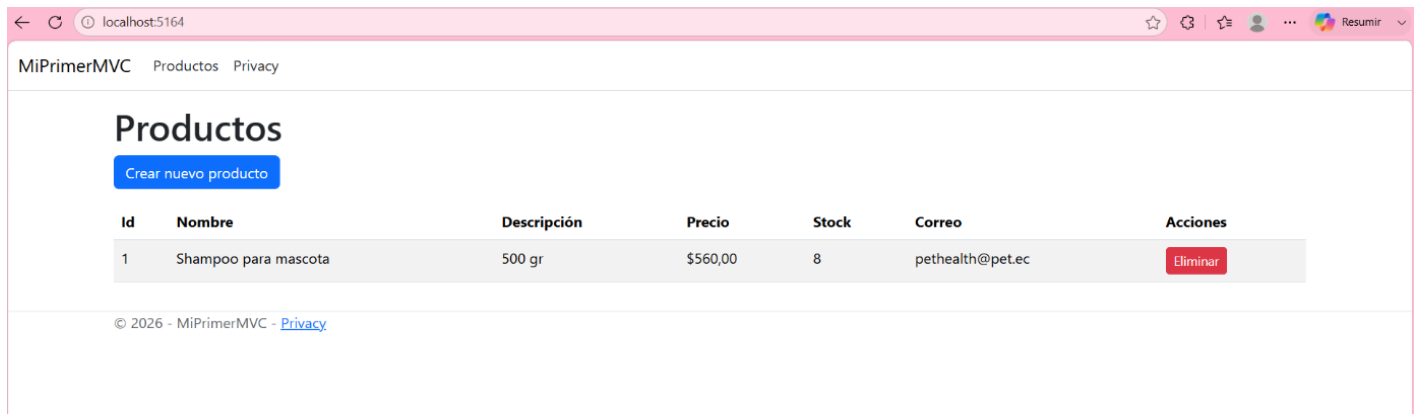


Figura 1: Listado de productos mostrando la tabla HTML con la columna Acciones

4. Acción Delete con confirmación POST y CSRF

Se implementó el patrón estándar de ASP.NET Core MVC para eliminar registros de forma segura: una acción GET que muestra una pantalla de confirmación (sin modificar datos), y una acción POST que ejecuta la eliminación real, protegida con [ValidateAntiForgeryToken]. De esta forma, un enlace o una solicitud GET nunca puede borrar un producto por sí sola.

4.1 Controlador: ProductoController

```
// GET: /Producto/Delete/5
[HttpGet]
public IActionResult Delete(int id)
{
    var producto = _productos.FirstOrDefault(p => p.Id == id);
    if (producto == null)
    {
        return NotFound();
    }

    return View(producto);
}

// POST: /Producto/Delete/5
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public IActionResult DeleteConfirmed(int id)
{
    var producto = _productos.FirstOrDefault(p => p.Id == id);
    if (producto != null)
    {
        _productos.Remove(producto);
    }

    return RedirectToAction(nameof(Index));
}
```

Listado 2: Controllers/ProductoController.cs — acciones Delete (GET) y DeleteConfirmed (POST)

El atributo [ActionName("Delete")] permite que el método C# se llame DeleteConfirmed (evitando duplicar el nombre Delete, ya usado por la versión GET) mientras que la ruta sigue

siendo `/Producto/Delete`. `[ValidateAntiForgeryToken]` obliga a que la petición POST incluya un token válido, generado previamente en la vista de confirmación.

4.2 Vista de confirmación: Delete.cshtml

```
@model MiPrimerMVC.Models.Producto

<h1>Eliminar Producto</h1>
<h3 class="text-danger">
    ¿Seguro que deseas eliminar este producto?
</h3>
<hr />
<dl class="row">
    <dt class="col-sm-4">Nombre</dt>
    <dd class="col-sm-8">@Model.Nombre</dd>
    ...
</dl>

@* El form tag helper genera automáticamente el campo
   __RequestVerificationToken, validado por [ValidateAntiForgeryToken]. *@
<form asp-action="Delete" method="post">
    <input type="hidden" asp-for="Id" />
    <button type="submit" class="btn btn-danger">Eliminar</button>
    <a asp-action="Index" class="btn btn-secondary">Cancelar</a>
</form>
```

Listado 3: Views/Producto/Delete.cshtml — confirmación con token CSRF automático

4.3 Resultados

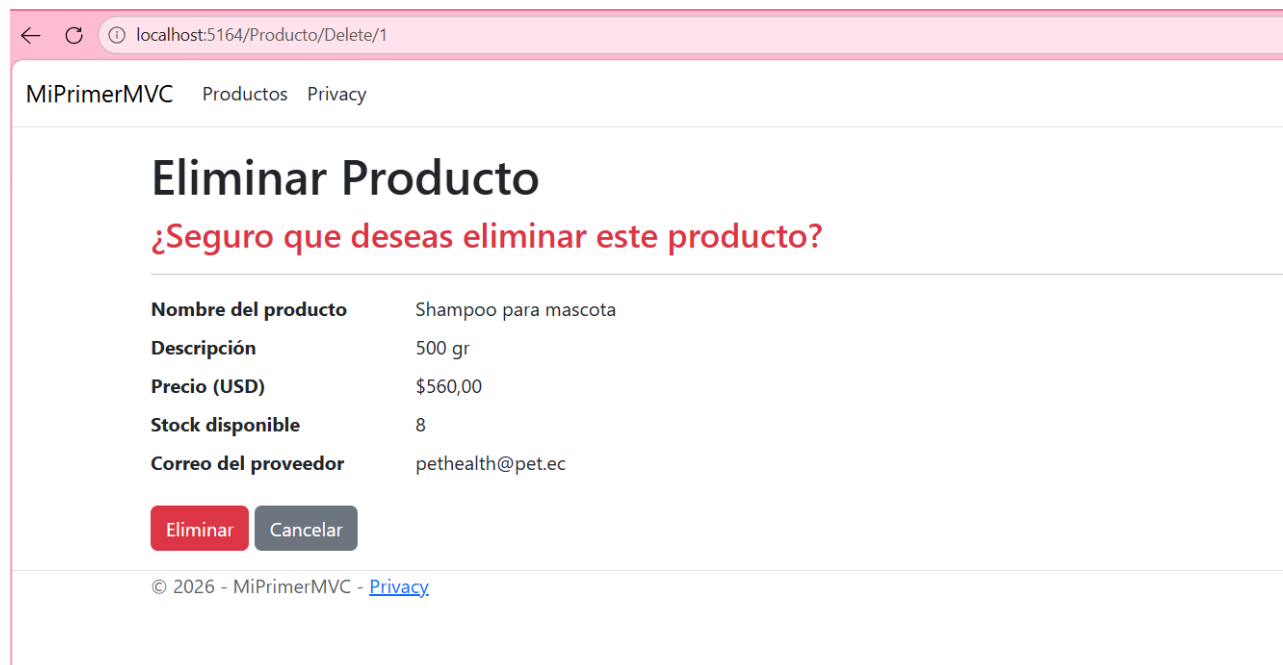


Figura 2: Pantalla de confirmación antes de eliminar un producto (`/Producto/Delete/{id}`)

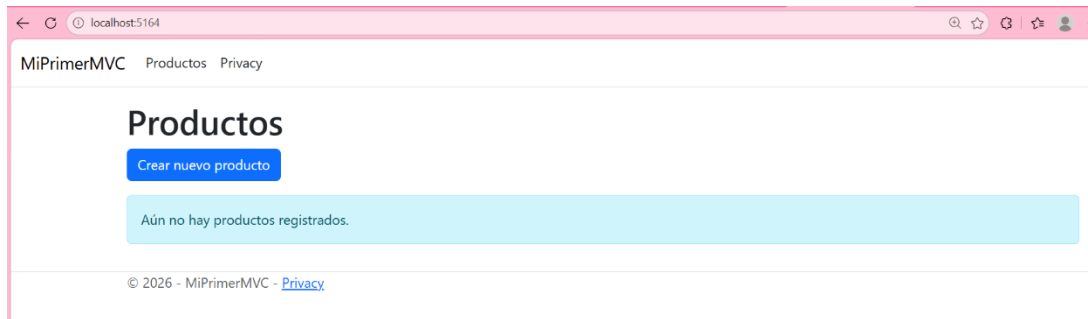


Figura 3: Listado actualizado mostrando que el producto fue eliminado correctamente

5. Validación en el cliente y en el servidor

El formulario de creación ya contaba con validación del lado del servidor mediante Data Annotations y ModelState.IsValid. Para esta tarea se activó además la validación del lado del cliente, de modo que ambas capas trabajen juntas: la del cliente ofrece una respuesta inmediata sin recargar la página, y la del servidor garantiza que los datos nunca se acepten si esa primera capa es evadida o está deshabilitada.

5.1 Validación del lado del cliente

Se agregó una sección de scripts en Create.cshtml que carga el parcial `_ValidationScriptsPartial` (jQuery + jQuery Validation Unobtrusive). Estos scripts leen los atributos `data-val-*` que los tag helpers `asp-for` generan automáticamente a partir de las Data Annotations del modelo, y muestran los mensajes de error en el navegador sin necesidad de enviar el formulario.

```
@section Scripts {  
    <partial name="_ValidationScriptsPartial" />  
}
```

Listado 4: Views/Producto/Create.cshtml — sección Scripts con validación de cliente

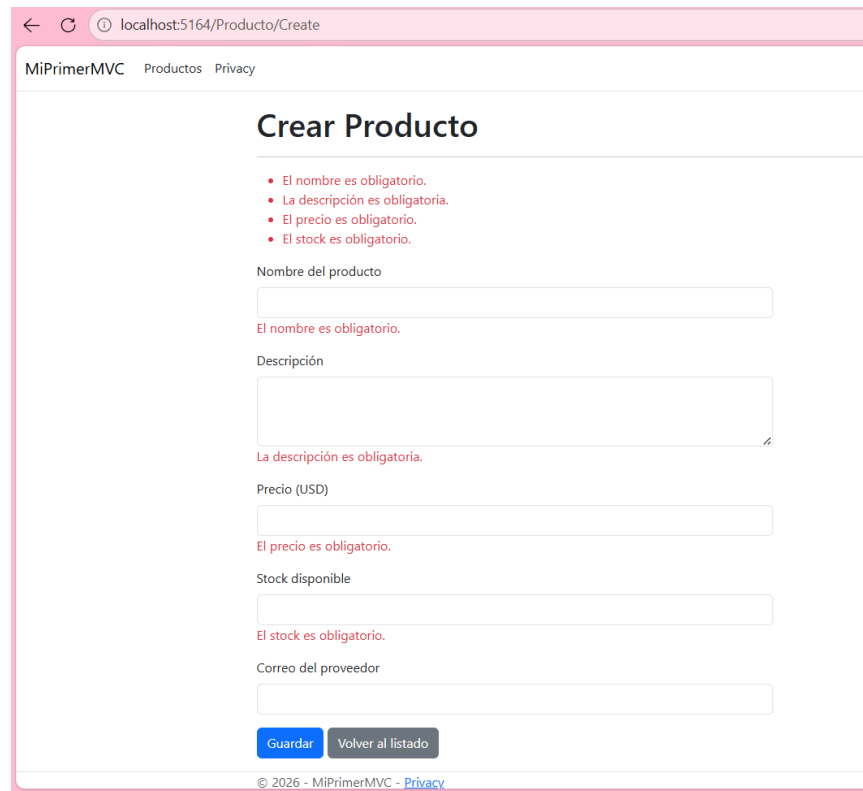
5.2 Validación del lado del servidor

La comprobación en el controlador no cambió respecto al proyecto base, y sigue siendo la última línea de defensa: aunque el navegador valide los campos, ModelState.IsValid se evalúa de nuevo en el servidor antes de guardar el producto.

```
[HttpPost]  
[ValidateAntiForgeryToken]  
public IActionResult Create(Producto producto)  
{  
    if (!ModelState.IsValid)  
    {  
        return View(producto);  
    }  
  
    producto.Id = _productos.Count + 1;  
    _productos.Add(producto);  
  
    return RedirectToAction(nameof(Index));  
}
```

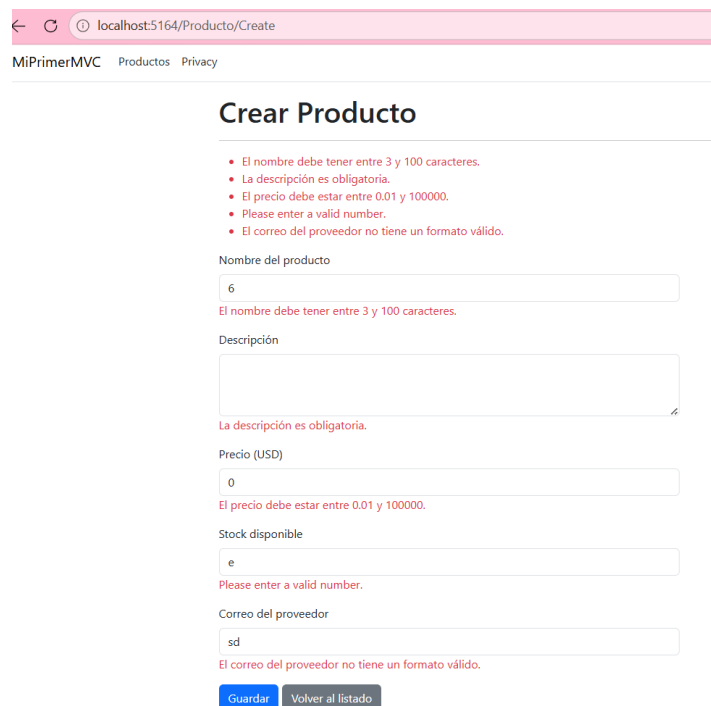
Listado 5: Controllers/ProductoController.cs — validación server-side con ModelState.IsValid

5.3 Resultados



A screenshot of a web browser at localhost:5164/Producto/Create. The page title is 'MiPrimerMVC' with links to 'Productos' and 'Privacy'. The main heading is 'Crear Producto'. A list of validation errors is shown in red: 'El nombre es obligatorio.', 'La descripción es obligatoria.', 'El precio es obligatorio.', and 'El stock es obligatorio.'. Below these are five input fields: 'Nombre del producto', 'Descripción', 'Precio (USD)', 'Stock disponible', and 'Correo del proveedor'. Each field has a red error message below it: 'El nombre es obligatorio.', 'La descripción es obligatoria.', 'El precio es obligatorio.', 'El stock es obligatorio.', and 'El stock es obligatorio.' (repeated). At the bottom are two buttons: 'Guardar' (blue) and 'Volver al listado' (grey). The footer shows '© 2026 - MiPrimerMVC - Privacy'.

Figura 4: Formulario enviado vacío mostrando los mensajes de error de las Data Annotations



A screenshot of the same web browser at localhost:5164/Producto/Create. The page title is 'MiPrimerMVC' with links to 'Productos' and 'Privacy'. The main heading is 'Crear Producto'. A list of validation errors is shown in red: 'El nombre debe tener entre 3 y 100 caracteres.', 'La descripción es obligatoria.', 'El precio debe estar entre 0.01 y 100000.', 'Please enter a valid number.', and 'El correo del proveedor no tiene un formato válido.'. Below these are five input fields: 'Nombre del producto' (containing '6'), 'Descripción', 'Precio (USD)' (containing '0'), 'Stock disponible' (containing 'e'), and 'Correo del proveedor' (containing 'sd'). Each field has a red error message below it: 'El nombre debe tener entre 3 y 100 caracteres.', 'La descripción es obligatoria.', 'El precio debe estar entre 0.01 y 100000.', 'Please enter a valid number.', and 'El correo del proveedor no tiene un formato válido.'. At the bottom are two buttons: 'Guardar' (blue) and 'Volver al listado' (grey). The footer shows '© 2026 - MiPrimerMVC - Privacy'.

Figura 5: Formulario con datos inválidos (precio igual a 0 o correo sin "@")

← localhost:5164/Producto/Create

MiPrimerMVC Productos Privacy

Crear Producto

Nombre del producto
Croquetas Procan

Descripción
20 kg

Precio (USD)
53.44

Stock disponible
10

Correo del proveedor
pronaca@pet.ec

Guardar Volver al listado

© 2026 - MiPrimerMVC - [Privacy](#)

Figura 6: Registro con datos válidos: la aplicación redirige al listado con el producto creado

6. Protección CSRF: AntiForgeryToken

Ambos formularios que modifican datos (Create y Delete) están protegidos contra ataques CSRF (Cross-Site Request Forgery). En Create, el token se genera de forma explícita con `@Html.AntiForgeryToken()`; en Delete, el propio form tag helper lo agrega automáticamente. En los dos casos, el método POST correspondiente del controlador valida ese token mediante el atributo `[ValidateAntiForgeryToken]`, rechazando cualquier solicitud que no lo incluya o cuyo token no coincida con el esperado por el servidor.

```

5 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 <title>Crear Producto - MiPrimerMVC</title>
7 <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" />
8 <link rel="stylesheet" href="/css/site.css" />
9 </head>
10 <body>
11 <header>
12 <nav class="navbar navbar-expand-sm navbar-light bg-white border-bottom box-shadow mb-3">
13 <div class="container-fluid">
14 <a class="navbar-brand" href="/Home">MiPrimerMVC</a>
15 <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
16 <ul class="navbar-nav flex-grow-1">
17 <li class="nav-item">
18 <a class="nav-link text-dark" href="/">Productos</a>
19 </li>
20 <li class="nav-item">
21 <a class="nav-link text-dark" href="/Home/Privacy">Privacy</a>
22 </li>
23 </ul>
24 </div>
25 </div>
26 </nav>
27 </header>
28 <div class="container">
29 <main role="main" class="pb-3">
30
31 <h1>Crear Producto</h1>
32 <hr />
33 <div class="row">
34 <div class="col-md-6">
35 <form method="post" action="/Producto/Create">
36 <input name="__RequestVerificationToken" type="hidden"
37 value="cf038cfe12w03h8MgP7Te0fEFCKXptryKY6rMH5euth915mTPyxmGH3a50P201AKQnUJHJZKElqSmcSE33ml77SVJmH97Kj3kQ1ldcyov1_I0wRCvRm3zc33yxnT0wXrtVJ3rGkyRCJGaqlmeGPrfBg" />
38 <div class="text-danger mb-3 validation-summary-valid" data-valmsg-summary="true"><ul style="display:none"></li>
39 </ul></div>
40
41 <div class="mb-3">
42 <label class="form-label" for="Nombre">Nombre del producto</label>
43 <input class="form-control" type="text" data-val="true" data-val-length="El nombre debe tener entre 3 y 100 caracteres." data-val-length-max="100" data-

```

Figura 7: Código fuente de la página /Producto/Create mostrando el campo oculto `__RequestVerificationToken`

7. Control de versiones: rama feature/practica-aspnet

Todo el desarrollo de esta tarea se realizó sobre una rama dedicada, separada de la rama principal que contiene la práctica en clase (GA). Esto permite mantener trazabilidad de los cambios y facilita una eventual revisión (pull request) antes de integrar la extensión a main.

```
git checkout -b feature/practica-aspnet

git add -A
git commit -m "TA: tabla con acciones, Delete con confirmacion y CSRF,"
git commit -m "validacion cliente+servidor, README actualizado"

git remote add origin https://github.com/Zaida-tm18/MiPrimerMVCExtendido.git
git push -u origin feature/practica-aspnet
```

Listado 6: comandos utilizados para crear y subir la rama feature/practica-aspnet

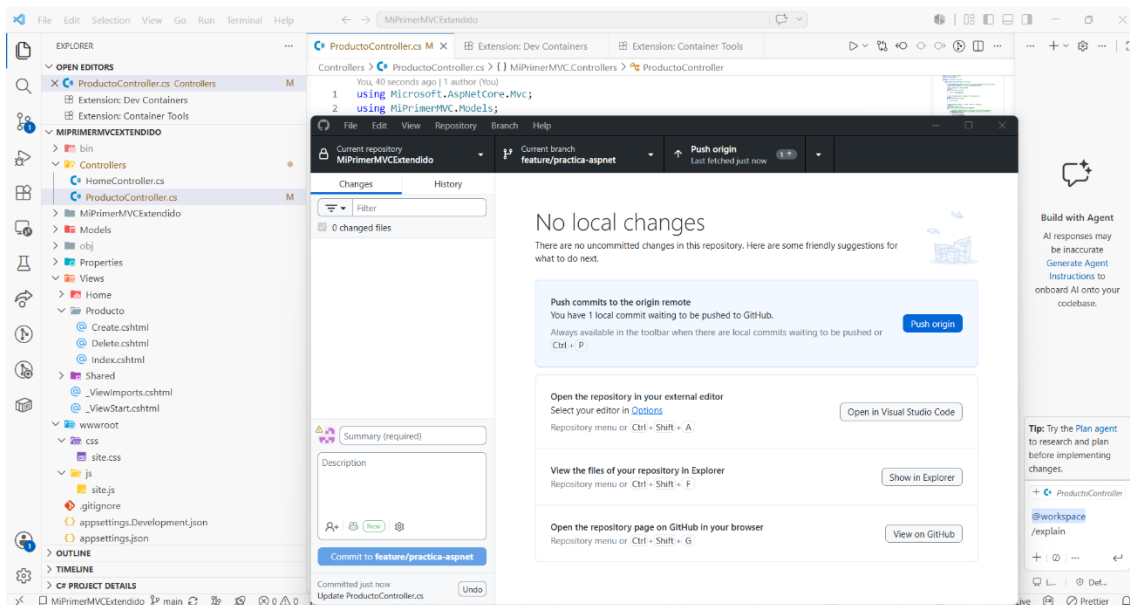


Figura 8: Terminal mostrando la rama feature/practica-aspnet activa y el push al repositorio remoto

8. Conclusiones

1. Se amplió el listado de productos con una columna de Acciones, manteniendo la tabla HTML ya existente y agregando el enlace de eliminación por fila.
2. Se implementó la eliminación de productos siguiendo el patrón GET de confirmación / POST de ejecución, evitando que una simple solicitud GET pueda borrar datos.
3. Ambas operaciones de escritura (Create y Delete) quedaron protegidas con [ValidateAntiForgeryToken], cerrando la puerta a ataques CSRF.
4. Se activó la validación del lado del cliente sin eliminar la validación del servidor, de modo que la aplicación sigue siendo segura incluso si JavaScript está deshabilitado.
5. El trabajo se organizó en la rama feature/practica-aspnet, separada de la rama principal, siguiendo buenas prácticas de control de versiones.